ARTI404 - Image Processing

Lab 3: Image Manipulations using OpenCV Library

Step 0: Install packages and download the sample image

In [1]:
```
%pip install -q opencv-python numpy matplotlib

import os

images_dir = os.path.join(os.getcwd(), 'images')
os.makedirs(images_dir, exist_ok=True)

url = 'https://raw.githubusercontent.com/PacktPublishing/OpenCV-3-x-with-Python-By-
dest = os.path.join(images_dir, 'input.jpg')

!curl -L -s -o "{dest}" "{url}"
size = os.path.getsize(dest) if os.path.exists(dest) else 0
print(f'input.jpg: {"OK" if size > 0 else "FAILED"} ({size} bytes) -> {dest}')
```

```
Note: you may need to restart the kernel to use updated packages.
input.jpg: OK (127967 bytes) -> c:\Users\dania\Image_Processing_Labs\Lab3\images\inp
ut.jpg
```

Step#1: Read an image using the OpenCV library

In [2]:
```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread(os.path.join('images', 'input.jpg'))

print(type(img))
print('shape:', img.shape)

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Input image')
plt.axis('off')
plt.show()
```

```
<class 'numpy.ndarray'>
shape: (458, 812, 3)
```



Step#2: Load an image in grayscale mode

In [3]:
```
gray_img = cv2.imread(os.path.join('images', 'input.jpg'), cv2.IMREAD_GRAYSCALE)

plt.imshow(gray_img, cmap='gray')
plt.title('Grayscale')
plt.axis('off')
plt.show()
```



Step#3: Save an image

In [4]:
```
os.makedirs('output', exist_ok=True)
cv2.imwrite(os.path.join('output', 'output.jpg'), gray_img)
print('Saved output/output.jpg')
```

```
Saved output/output.jpg
```

Step#4: Convert an image to different color spaces

In [5]:
```
img = cv2.imread(os.path.join('images', 'input.jpg'))
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.imshow(gray_img, cmap='gray')
plt.title('Grayscale image (via cvtColor)')
plt.axis('off')
plt.show()
```



Considering all the color spaces, there are around 190 conversion options available in OpenCV. The following prints the full list of color-related flags:

In [6]:
```
print([x for x in dir(cv2) if x.startswith('COLOR_')])
```

['COLOR_BAYER_BG2BGR', 'COLOR_BAYER_BG2BGRA', 'COLOR_BAYER_BG2BGR_EA', 'COLOR_BAYER_
BG2BGR_VNG', 'COLOR_BAYER_BG2GRAY', 'COLOR_BAYER_BG2RGB', 'COLOR_BAYER_BG2RGBA', 'CO
LOR_BAYER_BG2RGB_EA', 'COLOR_BAYER_BG2RGB_VNG', 'COLOR_BAYER_BGGR2BGR', 'COLOR_BAYER
_BGGR2BGRA', 'COLOR_BAYER_BGGR2BGR_EA', 'COLOR_BAYER_BGGR2BGR_VNG', 'COLOR_BAYER_BGG
R2GRAY', 'COLOR_BAYER_BGGR2RGB', 'COLOR_BAYER_BGGR2RGBA', 'COLOR_BAYER_BGGR2RGB_EA',
'COLOR_BAYER_BGGR2RGB_VNG', 'COLOR_BAYER_GB2BGR', 'COLOR_BAYER_GB2BGRA', 'COLOR_BAYE
R_GB2BGR_EA', 'COLOR_BAYER_GB2BGR_VNG', 'COLOR_BAYER_GB2GRAY', 'COLOR_BAYER_GB2RGB',
'COLOR_BAYER_GB2RGBA', 'COLOR_BAYER_GB2RGB_EA', 'COLOR_BAYER_GB2RGB_VNG', 'COLOR_BAY
ER_GBRG2BGR', 'COLOR_BAYER_GBRG2BGRA', 'COLOR_BAYER_GBRG2BGR_EA', 'COLOR_BAYER_GBRG2
BGR_VNG', 'COLOR_BAYER_GBRG2GRAY', 'COLOR_BAYER_GBRG2RGB', 'COLOR_BAYER_GBRG2RGBA',
'COLOR_BAYER_GBRG2RGB_EA', 'COLOR_BAYER_GBRG2RGB_VNG', 'COLOR_BAYER_GR2BGR', 'COLOR_
BAYER_GR2BGRA', 'COLOR_BAYER_GR2BGR_EA', 'COLOR_BAYER_GR2BGR_VNG', 'COLOR_BAYER_GR2G
RAY', 'COLOR_BAYER_GR2RGB', 'COLOR_BAYER_GR2RGBA', 'COLOR_BAYER_GR2RGB_EA', 'COLOR_B
AYER_GR2RGB_VNG', 'COLOR_BAYER_GRBG2BGR', 'COLOR_BAYER_GRBG2BGRA', 'COLOR_BAYER_GRBG
2BGR_EA', 'COLOR_BAYER_GRBG2BGR_VNG', 'COLOR_BAYER_GRBG2GRAY', 'COLOR_BAYER_GRBG2RG
B', 'COLOR_BAYER_GRBG2RGBA', 'COLOR_BAYER_GRBG2RGB_EA', 'COLOR_BAYER_GRBG2RGB_VNG',
'COLOR_BAYER_RG2BGR', 'COLOR_BAYER_RG2BGRA', 'COLOR_BAYER_RG2BGR_EA', 'COLOR_BAYER_R
G2BGR_VNG', 'COLOR_BAYER_RG2GRAY', 'COLOR_BAYER_RG2RGB', 'COLOR_BAYER_RG2RGBA', 'COL
OR_BAYER_RG2RGB_EA', 'COLOR_BAYER_RG2RGB_VNG', 'COLOR_BAYER_RGGB2BGR', 'COLOR_BAYER_
RGGB2BGRA', 'COLOR_BAYER_RGGB2BGR_EA', 'COLOR_BAYER_RGGB2BGR_VNG', 'COLOR_BAYER_RGGB
2GRAY', 'COLOR_BAYER_RGGB2RGB', 'COLOR_BAYER_RGGB2RGBA', 'COLOR_BAYER_RGGB2RGB_EA',
'COLOR_BAYER_RGGB2RGB_VNG', 'COLOR_BGR2BGR555', 'COLOR_BGR2BGR565', 'COLOR_BGR2BGR
A', 'COLOR_BGR2GRAY', 'COLOR_BGR2HLS', 'COLOR_BGR2HLS_FULL', 'COLOR_BGR2HSV', 'COLOR
_BGR2HSV_FULL', 'COLOR_BGR2LAB', 'COLOR_BGR2LUV', 'COLOR_BGR2Lab', 'COLOR_BGR2Luv',
'COLOR_BGR2RGB', 'COLOR_BGR2RGBA', 'COLOR_BGR2XYZ', 'COLOR_BGR2YCR_CB', 'COLOR_BGR2Y
CrCb', 'COLOR_BGR2YUV', 'COLOR_BGR2YUV_I420', 'COLOR_BGR2YUV_IYUV', 'COLOR_BGR2YUV_U
YNV', 'COLOR_BGR2YUV_UYVY', 'COLOR_BGR2YUV_Y422', 'COLOR_BGR2YUV_YUNV', 'COLOR_BGR2Y
UV_YUY2', 'COLOR_BGR2YUV_YUYV', 'COLOR_BGR2YUV_YV12', 'COLOR_BGR2YUV_YVYU', 'COLOR_B
GR5552BGR', 'COLOR_BGR5552BGRA', 'COLOR_BGR5552GRAY', 'COLOR_BGR5552RGB', 'COLOR_BGR
5552RGBA', 'COLOR_BGR5652BGR', 'COLOR_BGR5652BGRA', 'COLOR_BGR5652GRAY', 'COLOR_BGR5
652RGB', 'COLOR_BGR5652RGBA', 'COLOR_BGRA2BGR', 'COLOR_BGRA2BGR555', 'COLOR_BGRA2BGR
565', 'COLOR_BGRA2GRAY', 'COLOR_BGRA2RGB', 'COLOR_BGRA2RGBA', 'COLOR_BGRA2YUV_I420',
'COLOR_BGRA2YUV_IYUV', 'COLOR_BGRA2YUV_UYNV', 'COLOR_BGRA2YUV_UYVY', 'COLOR_BGRA2YUV
_Y422', 'COLOR_BGRA2YUV_YUNV', 'COLOR_BGRA2YUV_YUY2', 'COLOR_BGRA2YUV_YUYV', 'COLOR_
BGRA2YUV_YV12', 'COLOR_BGRA2YUV_YVYU', 'COLOR_BayerBG2BGR', 'COLOR_BayerBG2BGRA', 'C
OLOR_BayerBG2BGR_EA', 'COLOR_BayerBG2BGR_VNG', 'COLOR_BayerBG2GRAY', 'COLOR_BayerBG2
RGB', 'COLOR_BayerBG2RGBA', 'COLOR_BayerBG2RGB_EA', 'COLOR_BayerBG2RGB_VNG', 'COLOR_
BayerBGGR2BGR', 'COLOR_BayerBGGR2BGRA', 'COLOR_BayerBGGR2BGR_EA', 'COLOR_BayerBGGR2B
GR_VNG', 'COLOR_BayerBGGR2GRAY', 'COLOR_BayerBGGR2RGB', 'COLOR_BayerBGGR2RGBA', 'COL
OR_BayerBGGR2RGB_EA', 'COLOR_BayerBGGR2RGB_VNG', 'COLOR_BayerGB2BGR', 'COLOR_BayerGB
2BGRA', 'COLOR_BayerGB2BGR_EA', 'COLOR_BayerGB2BGR_VNG', 'COLOR_BayerGB2GRAY', 'COLO
R_BayerGB2RGB', 'COLOR_BayerGB2RGBA', 'COLOR_BayerGB2RGB_EA', 'COLOR_BayerGB2RGB_VN
G', 'COLOR_BayerGBRG2BGR', 'COLOR_BayerGBRG2BGRA', 'COLOR_BayerGBRG2BGR_EA', 'COLOR_
BayerGBRG2BGR_VNG', 'COLOR_BayerGBRG2GRAY', 'COLOR_BayerGBRG2RGB', 'COLOR_BayerGBRG2
RGBA', 'COLOR_BayerGBRG2RGB_EA', 'COLOR_BayerGBRG2RGB_VNG', 'COLOR_BayerGR2BGR', 'CO
LOR_BayerGR2BGRA', 'COLOR_BayerGR2BGR_EA', 'COLOR_BayerGR2BGR_VNG', 'COLOR_BayerGR2G
RAY', 'COLOR_BayerGR2RGB', 'COLOR_BayerGR2RGBA', 'COLOR_BayerGR2RGB_EA', 'COLOR_Baye
rGR2RGB_VNG', 'COLOR_BayerGRBG2BGR', 'COLOR_BayerGRBG2BGRA', 'COLOR_BayerGRBG2BGR_E
A', 'COLOR_BayerGRBG2BGR_VNG', 'COLOR_BayerGRBG2GRAY', 'COLOR_BayerGRBG2RGB', 'COLOR
_BayerGRBG2RGBA', 'COLOR_BayerGRBG2RGB_EA', 'COLOR_BayerGRBG2RGB_VNG', 'COLOR_BayerR
G2BGR', 'COLOR_BayerRG2BGRA', 'COLOR_BayerRG2BGR_EA', 'COLOR_BayerRG2BGR_VNG', 'COLO
R_BayerRG2GRAY', 'COLOR_BayerRG2RGB', 'COLOR_BayerRG2RGBA', 'COLOR_BayerRG2RGB_EA',
'COLOR_BayerRG2RGB_VNG', 'COLOR_BayerRGGB2BGR', 'COLOR_BayerRGGB2BGRA', 'COLOR_Bayer
RGGB2BGR_EA', 'COLOR_BayerRGGB2BGR_VNG', 'COLOR_BayerRGGB2GRAY', 'COLOR_BayerRGGB2RG
B', 'COLOR_BayerRGGB2RGBA', 'COLOR_BayerRGGB2RGB_EA', 'COLOR_BayerRGGB2RGB_VNG', 'CO
LOR_COLORCVT_MAX', 'COLOR_GRAY2BGR', 'COLOR_GRAY2BGR555', 'COLOR_GRAY2BGR565', 'COLO

```
R_GRAY2BGRA', 'COLOR_GRAY2RGB', 'COLOR_GRAY2RGBA', 'COLOR_HLS2BGR', 'COLOR_HLS2BGR_F
ULL', 'COLOR_HLS2RGB', 'COLOR_HLS2RGB_FULL', 'COLOR_HSV2BGR', 'COLOR_HSV2BGR_FULL',
'COLOR_HSV2RGB', 'COLOR_HSV2RGB_FULL', 'COLOR_LAB2BGR', 'COLOR_LAB2LBGR', 'COLOR_LAB
2LRGB', 'COLOR_LAB2RGB', 'COLOR_LBGR2LAB', 'COLOR_LBGR2LUV', 'COLOR_LBGR2Lab', 'COLO
R_LBGR2Luv', 'COLOR_LRGB2LAB', 'COLOR_LRGB2LUV', 'COLOR_LRGB2Lab', 'COLOR_LRGB2Luv',
'COLOR_LUV2BGR', 'COLOR_LUV2LBGR', 'COLOR_LUV2LRGB', 'COLOR_LUV2RGB', 'COLOR_Lab2BG
R', 'COLOR_Lab2LBGR', 'COLOR_Lab2LRGB', 'COLOR_Lab2RGB', 'COLOR_Luv2BGR', 'COLOR_Luv
2LBGR', 'COLOR_Luv2LRGB', 'COLOR_Luv2RGB', 'COLOR_M_RGBA2RGBA', 'COLOR_RGB2BGR', 'CO
LOR_RGB2BGR555', 'COLOR_RGB2BGR565', 'COLOR_RGB2BGRA', 'COLOR_RGB2GRAY', 'COLOR_RGB2
HLS', 'COLOR_RGB2HLS_FULL', 'COLOR_RGB2HSV', 'COLOR_RGB2HSV_FULL', 'COLOR_RGB2LAB',
'COLOR_RGB2LUV', 'COLOR_RGB2Lab', 'COLOR_RGB2Luv', 'COLOR_RGB2RGBA', 'COLOR_RGB2XY
Z', 'COLOR_RGB2YCR_CB', 'COLOR_RGB2YCrCb', 'COLOR_RGB2YUV', 'COLOR_RGB2YUV_I420', 'C
OLOR_RGB2YUV_IYUV', 'COLOR_RGB2YUV_UYNV', 'COLOR_RGB2YUV_UYVY', 'COLOR_RGB2YUV_Y42
2', 'COLOR_RGB2YUV_YUNV', 'COLOR_RGB2YUV_YUY2', 'COLOR_RGB2YUV_YUYV', 'COLOR_RGB2YUV
_YV12', 'COLOR_RGB2YUV_YVYU', 'COLOR_RGBA2BGR', 'COLOR_RGBA2BGR555', 'COLOR_RGBA2BGR
565', 'COLOR_RGBA2BGRA', 'COLOR_RGBA2GRAY', 'COLOR_RGBA2M_RGBA', 'COLOR_RGBA2RGB',
'COLOR_RGBA2YUV_I420', 'COLOR_RGBA2YUV_IYUV', 'COLOR_RGBA2YUV_UYNV', 'COLOR_RGBA2YUV
_UYVY', 'COLOR_RGBA2YUV_Y422', 'COLOR_RGBA2YUV_YUNV', 'COLOR_RGBA2YUV_YUY2', 'COLOR_
RGBA2YUV_YUYV', 'COLOR_RGBA2YUV_YV12', 'COLOR_RGBA2YUV_YVYU', 'COLOR_RGBA2mRGBA', 'C
OLOR_XYZ2BGR', 'COLOR_XYZ2RGB', 'COLOR_YCR_CB2BGR', 'COLOR_YCR_CB2RGB', 'COLOR_YCrCb
2BGR', 'COLOR_YCrCb2RGB', 'COLOR_YUV2BGR', 'COLOR_YUV2BGRA_I420', 'COLOR_YUV2BGRA_IY
UV', 'COLOR_YUV2BGRA_NV12', 'COLOR_YUV2BGRA_NV21', 'COLOR_YUV2BGRA_UYNV', 'COLOR_YUV
2BGRA_UYVY', 'COLOR_YUV2BGRA_Y422', 'COLOR_YUV2BGRA_YUNV', 'COLOR_YUV2BGRA_YUY2', 'C
OLOR_YUV2BGRA_YUYV', 'COLOR_YUV2BGRA_YV12', 'COLOR_YUV2BGRA_YVYU', 'COLOR_YUV2BGR_I4
20', 'COLOR_YUV2BGR_IYUV', 'COLOR_YUV2BGR_NV12', 'COLOR_YUV2BGR_NV21', 'COLOR_YUV2BG
R_UYNV', 'COLOR_YUV2BGR_UYVY', 'COLOR_YUV2BGR_Y422', 'COLOR_YUV2BGR_YUNV', 'COLOR_YU
V2BGR_YUY2', 'COLOR_YUV2BGR_YUYV', 'COLOR_YUV2BGR_YV12', 'COLOR_YUV2BGR_YVYU', 'COLO
R_YUV2GRAY_420', 'COLOR_YUV2GRAY_I420', 'COLOR_YUV2GRAY_IYUV', 'COLOR_YUV2GRAY_NV1
2', 'COLOR_YUV2GRAY_NV21', 'COLOR_YUV2GRAY_UYNV', 'COLOR_YUV2GRAY_UYVY', 'COLOR_YUV2
GRAY_Y422', 'COLOR_YUV2GRAY_YUNV', 'COLOR_YUV2GRAY_YUY2', 'COLOR_YUV2GRAY_YUYV', 'CO
LOR_YUV2GRAY_YV12', 'COLOR_YUV2GRAY_YVYU', 'COLOR_YUV2RGB', 'COLOR_YUV2RGBA_I420',
'COLOR_YUV2RGBA_IYUV', 'COLOR_YUV2RGBA_NV12', 'COLOR_YUV2RGBA_NV21', 'COLOR_YUV2RGBA
_UYNV', 'COLOR_YUV2RGBA_UYVY', 'COLOR_YUV2RGBA_Y422', 'COLOR_YUV2RGBA_YUNV', 'COLOR_
YUV2RGBA_YUY2', 'COLOR_YUV2RGBA_YUYV', 'COLOR_YUV2RGBA_YV12', 'COLOR_YUV2RGBA_YVYU',
'COLOR_YUV2RGB_I420', 'COLOR_YUV2RGB_IYUV', 'COLOR_YUV2RGB_NV12', 'COLOR_YUV2RGB_NV2
1', 'COLOR_YUV2RGB_UYNV', 'COLOR_YUV2RGB_UYVY', 'COLOR_YUV2RGB_Y422', 'COLOR_YUV2RGB
_YUNV', 'COLOR_YUV2RGB_YUY2', 'COLOR_YUV2RGB_YUYV', 'COLOR_YUV2RGB_YV12', 'COLOR_YUV
2RGB_YVYU', 'COLOR_YUV420P2BGR', 'COLOR_YUV420P2BGRA', 'COLOR_YUV420P2GRAY', 'COLOR_
YUV420P2RGB', 'COLOR_YUV420P2RGBA', 'COLOR_YUV420SP2BGR', 'COLOR_YUV420SP2BGRA', 'CO
LOR_YUV420SP2GRAY', 'COLOR_YUV420SP2RGB', 'COLOR_YUV420SP2RGBA', 'COLOR_YUV420p2BG
R', 'COLOR_YUV420p2BGRA', 'COLOR_YUV420p2GRAY', 'COLOR_YUV420p2RGB', 'COLOR_YUV420p2
RGBA', 'COLOR_YUV420sp2BGR', 'COLOR_YUV420sp2BGRA', 'COLOR_YUV420sp2GRAY', 'COLOR_YU
V420sp2RGB', 'COLOR_YUV420sp2RGBA', 'COLOR_mRGBA2RGBA']
```

Convert to YUV (Y: brightness, U: blue projection, V: red projection) and view each channel separately:

In [7]:
```
yuv_img = cv2.cvtColor(img, cv2.COLOR_BGR2YUV)

fig, axes = plt.subplots(1, 3, figsize=(14, 5))
axes[0].imshow(yuv_img[:, :, 0], cmap='gray'); axes[0].set_title('Y channel'); axes
axes[1].imshow(yuv_img[:, :, 1], cmap='gray'); axes[1].set_title('U channel'); axes
axes[2].imshow(yuv_img[:, :, 2], cmap='gray'); axes[2].set_title('V channel'); axes
plt.show()
```







Assessment

Task#1: Geometric transformations

1. Increase the size of the image.
2. Rotate the image by 120 degrees.
3. Perform a shear operation over the image.

In [8]:
```
import numpy as np

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
rows, cols = img_rgb.shape[:2]

# 1. Increase the size of the image (scale up by 1.5x)
enlarged = cv2.resize(img_rgb, None, fx=1.5, fy=1.5, interpolation=cv2.INTER_LINEAR

# 2. Rotate the image by 120 degrees
rotation_matrix = cv2.getRotationMatrix2D((cols / 2, rows / 2), 120, 1)
rotated = cv2.warpAffine(img_rgb, rotation_matrix, (cols, rows))

# 3. Shear operation (affine transform that skews the image)
src_points = np.float32([[0, 0], [cols - 1, 0], [0, rows - 1]])
dst_points = np.float32([[0, 0], [int(0.6 * (cols - 1)), 0], [int(0.4 * (cols - 1))
shear_matrix = cv2.getAffineTransform(src_points, dst_points)
sheared = cv2.warpAffine(img_rgb, shear_matrix, (cols, rows))

fig, axes = plt.subplots(1, 4, figsize=(18, 5))
axes[0].imshow(img_rgb); axes[0].set_title('Original'); axes[0].axis('off')
axes[1].imshow(enlarged); axes[1].set_title('Enlarged (1.5x)'); axes[1].axis('off')
axes[2].imshow(rotated); axes[2].set_title('Rotated 120°'); axes[2].axis('off')
axes[3].imshow(sheared); axes[3].set_title('Sheared'); axes[3].axis('off')
plt.show()
```



Task#2: Intensity transformations

1. Create a negative image.
2. Log transform (choose an appropriate constant for your image).

3. Power law / gamma transform (choose a gamma that increases the contrast).

In [9]:
```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)

# 1. Negative image
negative = 255 - gray

# 2. Log transform: s = c * log(1 + r)
c_log = 255 / np.log(1 + 255)
log_transformed = c_log * np.log(1 + gray)
log_transformed = np.clip(log_transformed, 0, 255).astype(np.uint8)

# 3. Power law (gamma) transform: s = c * r^gamma
gamma = 0.5  # gamma < 1 increases contrast/brightens the image
normalized = gray / 255.0
power_law = np.clip((normalized ** gamma) * 255, 0, 255).astype(np.uint8)

fig, axes = plt.subplots(1, 4, figsize=(18, 5))
axes[0].imshow(gray.astype(np.uint8), cmap='gray'); axes[0].set_title('Original (gr
axes[1].imshow(negative.astype(np.uint8), cmap='gray'); axes[1].set_title('Negative
axes[2].imshow(log_transformed, cmap='gray'); axes[2].set_title('Log transform'); a
axes[3].imshow(power_law, cmap='gray'); axes[3].set_title(f'Power law (gamma={gamma
plt.show()
```

